



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

**IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED
REASONING (RBR), AND CASE-BASED REASONING (CBR)**

Student Name	ELAZEGUI, LORENZO GABRIEL P.
Section:	AN42
Professor:	Ms. Crisola Tan
Date:	10/02/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
 - RBR was easier to implement because it only required simple IF-THEN rules. The conditions were easy to understand and the system directly gave an action based on the facts.
2. Which is more flexible in solving unstructured problems?
 - CBR is more flexible for unstructured problems because it can use previous cases as references instead of depending only on fixed rules.
3. Which is better for situations with continuously changing data?
 - CBR can be useful for continuously changing data because new cases can be added to the case base over time. This allows the system to use newer experiences when handling similar situations.
4. Examples of real-world applications for both.
 - RBR can be used in systems like automatic temperature control, troubleshooting systems, and simple decision-making programs. CBR can be used in customer support, medical case analysis, equipment troubleshooting, and recommendation systems.

Your Code - Laboratory Tasks ↓↓

```
facts = {
    "temperature": 32,
    "person_present": True,
    "window_open": False,
    "fan_on": False,
    "room_occupied": True
}

print("KNOWLEDGE REPRESENTATION")
print("=====")

print("Knowledge Base:")

for category, value in facts.items():
    print(category, ":", value)

def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the fan"

def rule_cold(facts):
    if facts["temperature"] < 20:
        return "Turn off the fan"

def rule_empty(facts):
    if not facts["person_present"]:
        return "Turn off the fan"

def inference_engine(facts):
    actions = []

    rules = [
        rule_hot,
        rule_cold,
        rule_empty
    ]

    for rule in rules:
        result = rule(facts)

        if result:
            actions.append(result)
```

```

    return actions

actions = inference_engine(facts)

print("\nRULE-BASED REASONING ACTIONS")
print("=====")

for action in actions:
    print("-", action)

case_base = [
    {
        "problem": {
            "temperature": 32,
            "person_present": True
        },
        "solution": "Turn on the fan"
    },
    {
        "problem": {
            "temperature": 28,
            "person_present": True
        },
        "solution": "Keep the fan off"
    },
    {
        "problem": {
            "temperature": 35,
            "person_present": True
        },
        "solution": "Turn on the fan"
    },
    {
        "problem": {
            "temperature": 18,
            "person_present": True
        },
        "solution": "Turn off the fan"
    },
    {
        "problem": {
            "temperature": 32,

```

```

        "person_present": False
    },
    "solution": "Turn off the fan"
}
]

```

```

new_problem = {
    "temperature": 32,
    "person_present": True
}

```

```

def calculate_similarity(new_problem, past_problem):
    score = 0

    if new_problem["temperature"] == past_problem["temperature"]:
        score += 1

    if new_problem["person_present"] == past_problem["person_present"]:
        score += 1

    return score

```

```

best_case = None
best_score = -1

```

```

print("\nCASE-BASED REASONING")
print("=====")

```

```

print("\nNew Problem:")
print(new_problem)

```

```

print("\nSimilarity Assessment:")

```

```

for case in case_base:
    score = calculate_similarity(
        new_problem,
        case["problem"]
    )

```

```

    print("Similarity Score:", score)

```

```

    if score > best_score:
        best_score = score

```

```
best_case = case

print("\nRetrieved Similar Case:")
print("Similarity Score:", best_score)
print("Previous Solution:", best_case["solution"])

reused_solution = best_case["solution"]

print("\nReused Solution:")
print(reused_solution)

revised_solution = reused_solution

print("\nRevised Solution:")
print(revised_solution)

new_case = {
    "problem": new_problem,
    "solution": revised_solution
}

case_base.append(new_case)

print("\nNew case saved successfully.")

print("\nUpdated Case Base:")

for case in case_base:
    print(case)
```